UE-05: Datenspeicherung analysieren

Team

- Jonathan Rusche
- Mikolaj Milczek

---

1. Methoden der Datenspeicherung

Die statische Analyse des APKs (Dekompilierung der DEX-Dateien) zeigt vier Datenspeichermethoden innerhalb des Namespaces `space.hitchhacker.guide`.

1.1 SharedPreferences (unverschlüsselt)

Klasse: `space.hitchhacker.guide.data.SharedPrefHelper`

Methode/Art: Standard Android SharedPreferences, unverschlüsselt, gespeichert als XML-Datei im privaten App-Verzeichnis.

Gespeicherte Daten (Keys):

Key (intern)	Beschreibung
`sharedPrefsUsername`	Benutzername des eingeloggten Nutzers
`sharedPrefsUrl`	URL des Hitchhacker-Guides (Standard oder angepasst)
`sharedPrefsManualUrl`	Manuell konfigurierte URL
`sharedPrefsFlag`	Boolean-Flag (z. B. Erstkonfiguration, Login-Status)

Nutzung in der App:

- Der Benutzername wird bei der Anmeldung (`LoginActivity`) gespeichert und im Profil-Fragment (`ProfileFragment`) angezeigt.
- Die URL wird im `GuideFragment` genutzt, um die Webansicht zu laden; über `UrlActivity` kann sie geändert werden.
- Das Flag steuert app-interne Zustände (z. B. ob ein manueller PIN gesetzt wurde).

1.2 EncryptedSharedPreferences (verschlüsselt)

Klasse: `space.hitchhacker.guide.data.EncSharedPrefHelper`

Methode/Art: `androidx.security.crypto.EncryptedSharedPreferences` mit `MasterKey` (AES256-GCM, gesichert im Android Keystore). Schlüssel und Werte werden separat verschlüsselt (AES256_SIV / AES256_GCM).

Gespeicherte Daten (Keys):

Key (intern)	Beschreibung
`encSharedPrefsPin`	Automatisch generierter PIN (4-stellig)
`encSharedPrefsManualPin`	Manuell gesetzter PIN (`MANUALPIN`)

Key (intern)	Beschreibung
`encSharedPrefsFlag` / `ENCFLAG`	Flag, ob manueller PIN aktiv ist

Die zugehörige Datei heißt `secret_shared_prefs`.

Nutzung in der App:

- Der PIN wird in `PinActivity` zur Authentifizierung des Nutzers beim App-Start abgefragt.
- `ControlPanel` (Vogon-Bereich) erlaubt das Setzen eines manuellen PINs.
- Die App prüft beim Start, ob ein manueller oder automatischer PIN hinterlegt ist, und leitet entsprechend um.

1.3 SQLite-Datenbank mit SQLCipher (verschlüsselt)

Klasse: `space.hitchhacker.guide.data.DiaryDatabase`

Methode/Art: Room-Datenbank auf Basis von SQLCipher (`net.zetetic.database.sqlcipher`). Die Datenbankdatei ist vollständig AES-verschlüsselt. Das `PRAGMA cipher_compatibility = 3` deutet auf SQLCipher v3-Kompatibilitätsmodus hin.

Datenbankschema:

```
CREATE TABLE diary (
    _id INTEGER PRIMARY KEY AUTOINCREMENT,
    content TEXT,
    date DATETIME DEFAULT CURRENT_TIMESTAMP
)
```

Gespeicherte Daten:

- Tagebucheinträge (Freitext + Zeitstempel) des Nutzers.
- Beim ersten Start werden vorausgefüllte Demo-Einträge eingefügt (`diary_entry1` bis `diary_entry4`).

Datenbankdatei: `diary.db`

Nutzung in der App:

- `DiaryFragment` liest alle Einträge und zeigt sie als Liste.
- `DiaryAddEntryFragment` fügt neue Einträge ein (`INSERT INTO diary ...`).
- `DiaryProvider` stellt die Daten als Content Provider bereit (Authority: `space.hitchhacker.guide.contentprovider`).

1.4 SQLite-Datenbank (Vogon/User, unverschlüsselt)

Klasse: `space.hitchhacker.guide.ui.vogon.UserDatabase`

Methode/Art: Standard SQLite-Datenbank (kein SQLCipher). Die Datenbankdatei ist **nicht verschlüsselt**.

Datenbankschema (aus Query-String):

```
SELECT * FROM users WHERE id = ?
```

Tabelle `users` mit mindestens den Spalten `id`, `password`, `hook`.

Gespeicherte Daten:

- Benutzerdaten inkl. Passwort-Hash oder Klartext-Passwort (der String `; password:` taucht im DEX auf).
- Aus dem Log-String `Copied users DB` geht hervor, dass die Datenbank unter bestimmten Bedingungen kopiert wird.

Datenbankdatei: `users.db`

Nutzung in der App:

- `VogonMain` und `ControlPanel` nutzen die Datenbank für den „Vogon"-Admin-Bereich.
- `UserProvider` exponiert Nutzerdaten als Content Provider (Authority: `space.hitchhacker.guide.vogon`).

1.5 Datei-Speicherung (internes Verzeichnis)

Klasse: `space.hitchhacker.guide.data.FileHelper`

Methode/Art: Direktes Schreiben in eine Textdatei über `openFileOutput` / `getFilesDir` (internes App-Verzeichnis).

Gespeicherte Daten:

- Suchanfragen bzw. Guide-relevante Textinhalte werden in `search.txt` geschrieben (`storeTextFile`-Methode).

Nutzung in der App:

- Das `GuideFragment` nutzt den FileHelper, um Suchanfragen lokal zu persistieren (z. B. für Offline-Zwecke oder Verlauf).

---

2. Dynamische Analyse – Dateisystempfade

> Die folgende Analyse beschreibt die erwarteten Pfade basierend auf dem Android-Speichermodell sowie den im Code gefundenen Dateinamen. Zur Verifikation sind die nachfolgenden `adb`-Befehle angegeben.

Vorbereitung

```
# App installieren
adb install hitchhacker_v3.apk

# Root-Shell öffnen
adb root
adb shell

# Als Root ins App-Verzeichnis wechseln
cd /data/data/space.hitchhacker.guide/
```

2.1 SharedPreferences (unverschlüsselt)

```
# Pfad
/data/data/space.hitchhacker.guide/shared_prefs/

# Datei(en) – Name entspricht dem im Code verwendeten Präferenz-Namen
```

```
ls /data/data/space.hitchhacker.guide/shared_prefs/

# Inhalt lesen (XML, im Klartext lesbar)
cat /data/data/space.hitchhacker.guide/shared_prefs/*.xml
```

Dateien: Eine XML-Datei mit den Keys `sharedPrefsUsername`, `sharedPrefsUrl`, `sharedPrefsManualUrl`, `sharedPrefsFlag`. Der genaue Dateiname entspricht dem beim `getSharedPreferences()`-Aufruf übergebenen String (z. B. `space.hitchhacker.guide_preferences.xml` oder ein explizit gewählter Name).

2.2 EncryptedSharedPreferences

```
# Pfad
/data/data/space.hitchhacker.guide/shared_prefs/

# Dateiname ist im Code als "secret_shared_prefs" definiert
cat /data/data/space.hitchhacker.guide/shared_prefs/secret_shared_prefs.xml
```

Datei: `secret_shared_prefs.xml` – Inhalt ist verschlüsselt (Base64-kodierte Ciphertext-Blobs, kein Klartext lesbar). Zusätzlich werden Tink-Keyset-Dateien für Schlüsselverwaltung angelegt:

```
# Tink-Keysets (automatisch von EncryptedSharedPreferences angelegt)
cat
/data/data/space.hitchhacker.guide/shared_prefs/__androidx_security_crypto_encrypted_prefs
_key_keyset__.xml
cat
/data/data/space.hitchhacker.guide/shared_prefs/__androidx_security_crypto_encrypted_prefs
_value_keyset__.xml
```

2.3 DiaryDatabase (SQLCipher)

```
# Pfad
/data/data/space.hitchhacker.guide/databases/

# Dateien
ls /data/data/space.hitchhacker.guide/databases/
# -> diary.db  diary.db-shm  diary.db-wal

# Datei ist verschlüsselt – direktes Lesen ergibt Binärdaten
hexdump -C /data/data/space.hitchhacker.guide/databases/diary.db | head -4
# Ausgabe: SQLCipher-Header (kein lesbarer "SQLite format 3"-Header)
```

2.4 UserDatabase (Standard SQLite)

```
# Pfad
/data/data/space.hitchhacker.guide/databases/

# Datei
ls /data/data/space.hitchhacker.guide/databases/
# -> users.db
```

```
# Im Klartext lesbar!
sqlite3 /data/data/space.hitchhacker.guide/databases/users.db
.tables
SELECT * FROM users;
```

Ergebnis: Die Tabelle `users` ist im Klartext auslesbar, inkl. Passwörter.

2.5 FileHelper (search.txt)

```
# Pfad
/data/data/space.hitchhacker.guide/files/

# Datei
cat /data/data/space.hitchhacker.guide/files/search.txt
```

Übersicht aller Pfade

Methode	Pfad & Dateiname
SharedPreferences (plain)	`/data/data/space.hitchhacker.guide/shared_prefs/<name>.xml`
EncryptedSharedPreferences	`/data/data/space.hitchhacker.guide/shared_prefs/secret_shared_prefs.xml`
DiaryDatabase (SQLCipher)	`/data/data/space.hitchhacker.guide/databases/diary.db`
UserDatabase (SQLite plain)	`/data/data/space.hitchhacker.guide/databases/users.db`
FileHelper	`/data/data/space.hitchhacker.guide/files/search.txt`

---

3. Sicherheitsbewertung

Was die App richtig macht

Verschlüsselung des Tagebuchs (DiaryDatabase):
Die sensibelsten Nutzerdaten – persönliche Tagebucheinträge – werden in einer SQLCipher-Datenbank gespeichert. Das ist eine gute Wahl, da der gesamte Datenbankinhalt auf Dateiebene AES-verschlüsselt ist.

Verschlüsselung des PINs (EncryptedSharedPreferences):
Der zur App-Sperrung verwendete PIN wird nicht im Klartext gespeichert, sondern in `EncryptedSharedPreferences` mit einem im Android Keystore gesicherten Schlüssel. Die Nutzung von `MasterKey` mit `AES256_GCM` entspricht aktuellem Best Practice.

Private Speicherorte:
Alle Daten liegen unter `/data/data/space.hitchhacker.guide/`, also im app-privaten Bereich. Ohne Root oder aktiviertes Backup sind diese Daten für andere Apps nicht direkt zugänglich.

Was verbessert werden sollte

`users.db` ist unverschlüsselt:
Die `UserDatabase` mit Tabelle `users` (inkl. Passwörtern) ist eine Standard-SQLite-Datenbank ohne Verschlüsselung. Ein Root-Angreifer oder eine bösartige App mit ausreichenden Rechten kann Benutzerdaten und Passwörter im Klartext auslesen. Diese Datenbank sollte ebenfalls mit SQLCipher verschlüsselt werden.

Passwörter vermutlich im Klartext:
Der im Code gefundene String `; password:` deutet darauf hin, dass Passwörter nicht gehasht, sondern als Klartext gespeichert werden. Passwörter sollten grundsätzlich nur als salted Hash (z. B. bcrypt oder Argon2) gespeichert werden, niemals als Klartext.

Benutzername in plain SharedPreferences:
Der Benutzername liegt unverschlüsselt in einer XML-Datei. Zwar ist er für sich genommen kein hochsensibles Datum, aber in Kombination mit anderen Informationen dennoch ein unnötiges Risiko. Alle persönlichen Daten sollten in `EncryptedSharedPreferences` gespeichert werden.

Content Provider ohne Berechtigungsprüfung:
Die App exponiert zwei Content Provider (`space.hitchhacker.guide.contentprovider` für das Tagebuch, `space.hitchhacker.guide.vogon` für Nutzerdaten). Ohne explizite `android:permission`-Einschränkungen im Manifest können andere Apps (auf gerooteten Geräten oder mit passenden Permissions) auf diese Provider zugreifen. Der `UserProvider` ist hier besonders kritisch, da er unverschlüsselte Benutzerdaten liefert.

SQLCipher v3-Kompatibilitätsmodus:
Das Pragma `cipher_compatibility = 3` aktiviert einen Rückwärtskompatibilitätsmodus zu SQLCipher v3, der ältere (schwächere) Standardparameter verwendet. Für neue Datenbanken sollte der aktuelle SQLCipher v4-Standard (256-Bit AES, PBKDF2 mit 256.000 Iterationen) ohne Kompatibilitätsmodus genutzt werden.

`search.txt` im Klartext:
Suchanfragen im Klartext abgelegt – je nach Inhalt der Suchanfragen könnte dies Nutzungsverhalten und sensible Begriffe preisgeben. Auch diese Datei sollte verschlüsselt werden (z. B. via `EncryptedFile` aus `androidx.security.crypto`).

Zusammenfassung

Datenspeicherung	Verschlüsselt	Bewertung
SharedPreferences (Nutzername, URL)	❌ Nein	Verbesserungsbedarf
EncryptedSharedPreferences (PIN)	☑ Ja	Gut
DiaryDatabase / diary.db	☑ Ja (SQLCipher)	Gut
UserDatabase / users.db	❌ Nein	Kritisch
FileHelper / search.txt	❌ Nein	Verbesserungsbedarf